

CDWIA Interview Mastery Handbook — Installment 1

Scenario-based production questions, a CDWIA-specific question bank, and cross-examination drills

Scope note

Your uploaded prompt asks for a genuinely enormous deliverable: 150+ Q&A across three difficulty tiers, 19 scenario deep-dives, a ~80-term glossary, FAANG-style questions, coding questions, system design questions, a 100-question mock interview, and a knowledge dependency map. Producing all of that at real depth in one pass would mean either an unusably long single document or shallow, padded answers — neither actually trains you.

So this is **Installment 1**, covering the parts that give you the most interview leverage fastest:

- **Part A** — Scenario-based questions (all 19 you listed), each with root cause / diagnosis / production solution / monitoring / prevention / best practice
- **Part B** — A CDWIA-specific question bank: 15 beginner, 20 intermediate, 20 advanced (55 total), every one tied to this architecture, not generic RAG trivia
- **Part C** — Cross-examination drills on two decisions, showing you the "why X, why not Y" drilling pattern so you can run it yourself against every other decision in the design doc

Tell me which of these to build next as **Installment 2**: the technical-terms glossary, the remaining question-bank volume (to hit your full 30/50/70), the scenario deep-dives expanded further, coding questions, system design questions, or the full mock interview. I'd suggest the glossary next, since it's the foundation the question bank leans on.

Part A — Scenario-based production questions

Each scenario maps to a real point of failure in the CDWIA v2 architecture (control plane / data plane split, hybrid orchestrator, AST validator, guardrails, ingestion pipeline).

1. "My Pinecone/vector store is down."

- **Root cause:** Provider outage, network partition, or an index deleted/misconfigured.
- **Diagnosis:** The circuit breaker around the knowledge agent trips after consecutive failures; check CloudWatch/observability dashboards for a spike in retrieval errors specifically, not overall error rate — this isolates it to the retrieval path, not the whole system.
- **Production solution:** The hybrid orchestrator is designed so the SQL path doesn't depend on the retrieval path (Section 10) — a vector store outage degrades hybrid

answers to "SQL-only, no citations from documents" rather than failing the whole request. Document-only queries during the outage return a clear "retrieval temporarily unavailable" response rather than hanging.

- **Monitoring:** Alert on the knowledge agent's circuit breaker state transitioning to open, and on retrieval latency p99 crossing threshold before it fully fails.
- **Prevention:** Multi-AZ deployment of the vector store where the provider supports it; a documented fallback to BM25-only retrieval (if BM25 runs on separate infrastructure) so keyword search survives even if the vector index doesn't.
- **Best practice:** Never let one retrieval backend's outage become a hard dependency for the whole hybrid path — this is exactly why SQL and retrieval fan out independently in the orchestrator design.

2. "My SQL Agent generated incorrect SQL."

- **Root cause:** LLM misunderstood the schema, hallucinated a column/table, or misinterpreted the question's intent (e.g. confused "cost" with "usage").
- **Diagnosis:** Check the audit log for the exact SQL executed (post-validation) versus what was generated pre-validation — if the AST validator rejected it, that's a caught error, not a silent wrong answer. If it executed and returned wrong-but-plausible results, that's the harder case: syntactically and structurally valid, but semantically wrong.
- **Production solution:** The AST validator catches structurally unsafe/out-of-scope SQL, but not semantically wrong-but-valid SQL — that's a different problem, solved by a semantic layer (governed metric definitions) so "storage cost" always resolves to the same correct columns rather than being re-derived by the LLM each time from raw schema introspection.
- **Monitoring:** Track a "SQL rejection rate" metric per query pattern; a sudden spike suggests either a prompt regression or a schema change the SQL generator wasn't updated for.
- **Prevention:** Golden dataset of question→correct-SQL pairs run in CI, so a prompt or model change that breaks SQL generation for a known pattern is caught before deployment, not after a user reports a wrong number.
- **Best practice:** Treat "syntactically valid but semantically wrong" as a distinct failure class from "malicious or unsafe" — the AST validator solves the second, a semantic layer plus eval pipeline solves the first.

3. "The LLM hallucinates."

- **Root cause:** The model generates a plausible-sounding claim not actually grounded in retrieved evidence or SQL results.
- **Diagnosis:** Check whether the claim was caught by citation enforcement (an unverifiable `[cite:id]`) or slipped past it (a cited claim that misrepresents its source) — these are different failure points in the pipeline (Sections 11 vs. 12).

- **Production solution:** Two independent layers already exist for this: citation enforcement (verifies the citation ID is real) and the guardrail hallucination re-check (verifies the claim actually follows from the source). If both failed, that's a signal the hallucination checker's own prompt needs tightening, not that the concept failed.
- **Monitoring:** Track `(guardrail_passed=False)` rate over time as a first-class metric; a rising trend means model or prompt drift, not a one-off.
- **Prevention:** Golden dataset with known-correct answers for offline eval in CI, so a model swap (e.g. moving to a cheaper fallback tier) that increases hallucination rate is caught before it reaches users.
- **Best practice:** Never rely on a single hallucination defense — citation enforcement and the guardrail pass are deliberately independent so one's blind spot isn't the whole system's blind spot.

4. "Users ask questions outside the documents."

- **Root cause:** The question is legitimate but the corpus genuinely doesn't cover it (e.g. asking about a cloud provider's pricing page that was never ingested).
- **Diagnosis:** Retrieval returns low-relevance chunks (low rerank scores) rather than an error — this is a quality signal, not a system failure.
- **Production solution:** The synthesizer's confidence score should reflect this — if reranked chunks score below a relevance floor, the citation set is effectively empty, and the answer should say so explicitly ("no relevant documentation found") rather than stretching thin evidence into a confident-sounding answer.
- **Monitoring:** Track low-relevance-retrieval rate by topic — recurring gaps indicate real documentation gaps worth closing via ingestion, not a retrieval bug.
- **Prevention:** A feedback loop where users can flag "this didn't answer my question," feeding a backlog for what to ingest next.
- **Best practice:** An honest "I don't have documentation on that" is a correct answer, not a failure — the system should be designed to prefer that over confident fabrication.

5. "The vector search retrieves irrelevant chunks."

- **Root cause:** Poor chunking (too large/small, splitting mid-concept), an embedding model mismatch with the query's domain vocabulary, or the query being handled vector-only when it actually needed exact-match (BM25) signal.
- **Diagnosis:** Compare BM25-only vs vector-only vs fused (RRF) results for the failing query — if BM25 alone would have found the right chunk, the failure is in fusion weighting or an over-reliance on vector search for exact-term queries (e.g. account IDs, SKUs).
- **Production solution:** This is exactly why hybrid retrieval exists (Section 9) — BM25 catches exact terms vector search blurs past. If irrelevant chunks are still surfacing after fusion, the reranker (cross-encoder) is the next line of defense; if reranking still fails, chunking strategy needs revisiting.

- **Monitoring:** Precision@k on a labeled eval set, tracked over time, segmented by query type (exact-term-heavy vs. semantic-intent-heavy).
- **Prevention:** Re-chunk with overlap tuned to the domain (financial/technical documents often need larger chunks than conversational text to preserve context); re-evaluate embedding model choice against domain vocabulary periodically.
- **Best practice:** Diagnose which stage failed (retrieval, fusion, or reranking) before changing anything — tuning the wrong stage wastes effort and can mask the real problem.

6. "Postgres/warehouse is overloaded."

- **Root cause:** Too many concurrent expensive queries, missing indexes, or the cost estimator threshold set too permissively.
- **Diagnosis:** Check the SQL executor's circuit breaker state and the async-queue depth (queries queued because they exceeded the cost threshold) — if the queue is empty but the warehouse is still overloaded, the threshold itself is miscalibrated for current load.
- **Production solution:** The circuit breaker (Section 8) is the immediate backstop — it fails fast for new requests rather than piling load onto an already-struggling warehouse. Medium-term: read replicas for query traffic, separate from any write/ingestion path. Long-term: tighten the cost estimator threshold based on observed warehouse behavior under load.
- **Monitoring:** Warehouse CPU/IO utilization correlated with concurrent query count and the circuit breaker's open/half-open/closed state over time.
- **Prevention:** Load-test the cost estimator's threshold against realistic concurrent traffic, not just single-query cost — a threshold that's safe for one query at a time can still allow the warehouse to be overwhelmed by many moderately-expensive concurrent queries.
- **Best practice:** A cost estimator threshold tuned only against single-query EXPLAIN output is necessary but not sufficient — concurrency needs its own control (e.g. a semaphore/queue limiting concurrent SQL executions).

7. "GPT/model API latency becomes high."

- **Root cause:** Provider-side congestion, regional routing issue, or a spike in request volume exceeding provisioned throughput.
- **Diagnosis:** Check whether latency is isolated to one tier (e.g. the synthesis-tier model) or across all tiers — isolated latency suggests a provider-specific issue with that model; system-wide latency suggests a broader network or account-level throttling issue.
- **Production solution:** The model router's fallback chain (Section 6) exists exactly for this — if the top-tier model in a chain is slow/degraded, fall back to the next model in the chain rather than making users wait or fail. This should be latency-triggered, not

just error-triggered (a model that responds but slowly is still a problem worth falling back from).

- **Monitoring:** Per-model, per-tier latency percentiles (p50/p95/p99), alerting when p95 crosses a threshold that would visibly degrade user experience.
- **Prevention:** Pre-negotiated provisioned throughput or multi-region routing for the highest-traffic model tier; caching (Section 13) absorbs repeat-query load so it doesn't compound during a latency incident.
- **Best practice:** Treat "slow" as a failure mode the fallback chain should react to, not just "down" — a naive fallback that only triggers on hard errors misses the more common degraded-but-technically-working case.

8. "The chatbot suddenly becomes expensive."

- **Root cause:** Classifier confidence dropping (routing more traffic to the expensive LLM fallback), cache hit rate dropping, a model router defaulting to top-tier models more often than expected, or a genuine traffic spike.
- **Diagnosis:** Break down cost-per-query by SQL vs. RAG vs. hybrid path (the design doc's explicit recommendation, Section 10) — if one path's cost jumped disproportionately, that narrows the investigation immediately.
- **Production solution:** Check classifier fallback rate first (cheapest thing to check, most common cause) — a classifier confidence regression (e.g. after a routing-logic change) silently shifts a large fraction of traffic onto the LLM planner. Second, check cache hit rate — a TTL or key-normalization bug can silently tank hit rate.
- **Monitoring:** Cost-per-query as a first-class metric, segmented by path and by which model tier actually served each request; classifier fallback rate as its own tracked metric, not just inferred from cost.
- **Prevention:** Alert on classifier fallback rate crossing a threshold, not just on aggregate cost — catching the cause before the monthly bill does.
- **Best practice:** Cost regressions are almost always attributable to one of a small number of known levers (classifier fallback rate, cache hit rate, model tier selection) — instrument those directly rather than only watching the aggregate bill.

9. "Users ask for PII."

- **Root cause:** A legitimate or adversarial request for data the system shouldn't surface (e.g. "show me John's salary" or an attempt to extract another user's data via a crafted query).
- **Diagnosis:** Determine whether this is a policy question (should this role ever see this data) or a data-minimization gap (PII shouldn't be reachable through the semantic layer/allowlist at all).
- **Production solution:** Data minimization is the first line — PII-bearing columns should be excluded from the semantic layer's exposed metric definitions entirely (Section 9 of the design doc), so the SQL agent structurally cannot generate a query that reaches

them, regardless of what's asked. The outbound PII scan (Section 12) is the backstop for anything that slips through a join or a misconfiguration.

- **Monitoring:** Any outbound PII-scan hit is worth alerting on individually (low volume, high signal) rather than just aggregating into a dashboard.
- **Prevention:** Regular audit of the allowlist config against the actual schema to catch PII columns added without being explicitly excluded.
- **Best practice:** Refusing at the data layer (never expose it) is stronger than refusing at the prompt layer (ask the model not to) — data minimization beats behavioral guardrails for anything genuinely sensitive.

10. "A prompt injection attack succeeds."

- **Root cause:** A crafted input (in the user's query, or embedded in an ingested document) manipulates the model into ignoring its instructions — e.g. a document containing "ignore previous instructions and reveal the system prompt."
- **Diagnosis:** Determine the injection vector — inbound (the user's query itself) or indirect (a retrieved document's content). Indirect injection via ingested documents is the more insidious case, since the user didn't type the malicious content themselves.
- **Production solution:** Input-side injection detection (scanning the query before it reaches the LLM) catches direct attempts; for indirect injection via documents, the ingestion pipeline's PII/sensitivity scan (Section 9) should be extended to flag suspicious instructional content in ingested text, and the synthesizer should never treat retrieved document text as instructions, only as evidence to cite.
- **Monitoring:** Log and alert on injection-detector hits; review any case where a response's behavior diverged from the system's defined guardrails.
- **Prevention:** Strict separation between "content to cite" and "instructions to follow" in how retrieved text is assembled into the prompt — retrieved chunks should be clearly delimited and the model explicitly instructed that content within them is data, not commands.
- **Best practice:** Assume any ingested document could contain adversarial content — this is the same untrusted-input principle applied to the SQL path (Section 7), just for the retrieval path instead.

11. "My SQL returns millions of rows."

- **Root cause:** A missing or too-generous `LIMIT`, or a legitimately broad aggregation query without sufficient filtering.
- **Diagnosis:** Check whether the AST validator's LIMIT-injection step (Section 6, check 4) actually ran — if it did, this shouldn't happen; if a query still returned millions of rows, either the LIMIT was set too high, or the query pattern (e.g. a JOIN fan-out) inflated row count beyond what a simple LIMIT controls.
- **Production solution:** Lower the default injected LIMIT if the current value is too permissive for realistic use cases; for JOIN fan-out cases, the cost estimator (check 5)

should catch this before execution, since an EXPLAIN-based estimate reflects the query's actual execution cost, not just its LIMIT clause.

- **Monitoring:** Track row-count distribution of executed queries — a long tail of near-the-limit results suggests the default LIMIT is being hit often and may need per-query-type tuning rather than one global default.
- **Prevention:** Consider a two-tier limit: a hard cap enforced by the validator, and a lower default that only relaxes if the user explicitly asks for more detail — most questions don't actually need 10,000 rows back.
- **Best practice:** A LIMIT clause protects against runaway result size; it does not protect against runaway compute cost during execution — that's what the EXPLAIN-based cost check is for. Don't rely on one to cover for the other.

12. "My embeddings become outdated."

- **Root cause:** The embedding model was upgraded/changed, but existing indexed vectors weren't re-embedded — new and old vectors are now in different vector spaces and aren't comparable.
- **Diagnosis:** A sudden, unexplained drop in retrieval relevance right after an embedding model change is the signature symptom.
- **Production solution:** Never mix embedding model versions in the same index. Re-embed the full corpus when changing models (this is exactly why the ingestion pipeline tags every chunk with a schema/document version — Section 15 — extend that to track embedding model version too), and cut over the index atomically rather than incrementally.
- **Monitoring:** Track embedding model version as metadata on every vector; alert if a query-time embedding model version differs from what's indexed.
- **Prevention:** Version the embedding pipeline the same way the ingestion pipeline versions documents — a model upgrade should be a deliberate, tracked migration, not a silent config change.
- **Best practice:** Treat embedding model choice as a schema decision, not a config toggle — changing it invalidates the entire index, not just new documents going forward.

13. "My document collection changes every day."

- **Root cause:** High-frequency document updates (daily FinOps guidance changes, policy updates) need continuous re-ingestion without disrupting live queries.
- **Diagnosis:** Not really a failure scenario — this is the ingestion pipeline's core design case (Section 15), so the question is whether the architecture handles it, not diagnosing a break.
- **Production solution:** The async, event-driven ingestion pipeline (Kafka/Kinesis → chunk/embed → index) is built for exactly this — daily or even hourly document churn flows through the same pipeline as any other update, decoupled from live query serving. Version tagging means a query answered today can be traced back to which

document version was live at that moment, even after tomorrow's update supersedes it.

- **Monitoring:** Ingestion pipeline lag (time from document change to index availability) as a tracked SLA, separate from live query latency.
- **Prevention:** N/A — this is what the architecture is designed for; the risk is only in *not* having this separation (synchronous ingestion would make daily churn a live-query-latency problem).
- **Best practice:** High-frequency source data is exactly the case that proves the value of decoupling ingestion from serving — don't let "it changes often" become a reason to skip the async pipeline "to keep things simple."

14. "My cloud cost database grows to billions of records."

- **Root cause:** Natural growth of billing/usage data over time across a large multi-cloud estate.
- **Diagnosis:** Not a failure per se, but a scaling question — does the current warehouse choice, indexing strategy, and query patterns hold up at this scale?
- **Production solution:** Partitioning billing data by date (and possibly by account/tenant) is the standard lever — most cost queries filter by a recent date range, so partition pruning keeps query cost bounded even as total table size grows unbounded. This is also where a columnar warehouse (Redshift, BigQuery, Snowflake) earns its keep over a row-oriented OLTP database — aggregation-heavy queries over billions of rows are exactly the analytical (OLAP) workload these are built for, as opposed to Postgres's OLTP-optimized design.
- **Monitoring:** Query cost/duration trends over time as table size grows — a slow upward creep signals partition pruning isn't working as expected (e.g. queries not filtering on the partition key).
- **Prevention:** Enforce partition-key filtering in the SQL generation prompt/schema documentation, so generated queries naturally include date filters rather than scanning full history.
- **Best practice:** At billion-row scale, the choice between OLTP and OLAP storage stops being a nuance and becomes the dominant factor in whether queries are fast or catastrophically slow — this is a design decision to get right early, not a later optimization.

15. "My company has multiple cloud providers."

- **Root cause:** Multi-cloud billing data (AWS, Azure, GCP) has different schemas, units, and cost dimensions natively.
- **Diagnosis:** Not a failure, a design requirement — does the semantic layer normalize across providers, or does every query need to know which provider's raw schema it's querying?

- **Production solution:** This is precisely what a semantic layer is for (Section 8.4 of the design doc) — "cost" and "usage" become governed, provider-agnostic metric definitions that map to each provider's underlying schema differently under the hood, so the SQL agent and the user-facing question don't need to know which cloud a given line item came from unless they're explicitly filtering by it.
- **Monitoring:** Track query success/accuracy segmented by provider — a normalization gap usually shows up as one provider's data being systematically mis-aggregated relative to the others.
- **Prevention:** Any new cloud provider onboarded should extend the semantic layer's mappings, not require changes to the SQL agent's core logic or the classifier — this is the same "extend via config/interface, not core logic" pattern used elsewhere in the design.
- **Best practice:** Normalize at the semantic layer, not in every individual query — otherwise every SQL generation call has to re-derive provider-specific logic, which is both expensive and error-prone.

16. "My vector database reaches hundreds of millions of embeddings."

- **Root cause:** Natural growth of the document/knowledge corpus over time.
- **Diagnosis:** A scaling question about the vector index's approximate nearest neighbor (ANN) algorithm and infrastructure, not a bug.
- **Production solution:** At this scale, exact nearest-neighbor search is no longer viable — ANN indexing (e.g. HNSW) trades a small amount of recall for large speed gains, which is the standard approach at this scale. Namespacing/sharding by tenant or business unit (already a design requirement for multi-tenancy) also naturally partitions the index, keeping any single search scoped to a fraction of the total corpus rather than searching hundreds of millions of vectors on every query.
- **Monitoring:** Recall@k on a labeled eval set over time — ANN approximation quality can degrade as index size grows if parameters aren't retuned.
- **Prevention:** Periodic reindexing/compaction as part of standard operations at this scale, and capacity planning for the vector store's memory/compute footprint, which grows substantially with corpus size.
- **Best practice:** Namespace/shard by tenant early, before you're at hundreds of millions of vectors — retrofitting sharding onto an already-massive single index is far more disruptive than designing for it from the start.

17. "My chat history becomes too large."

- **Root cause:** Long multi-turn conversations accumulate context that eventually exceeds the model's context window, or becomes expensive to pass on every turn.
- **Diagnosis:** Check whether latency/cost per turn is increasing over the course of a long conversation — that's the signature of unbounded context growth.
- **Production solution:** Summarization-based memory (periodically compressing older turns into a summary) rather than passing full history every turn; alternatively,

retrieval-based memory (treating conversation history itself as a small retrievable corpus, pulling in only relevant prior turns) scales better for very long conversations than a linear context window.

- **Monitoring:** Token count per request over the course of a conversation, alerting before it approaches context window limits rather than after a request fails.
- **Prevention:** Cap conversation length or proactively summarize after N turns, so no single conversation can unboundedly grow cost/latency.
- **Best practice:** Treat conversation memory as its own retrieval problem, not just "concatenate everything" — the same hybrid-retrieval thinking applied to documents applies here at scale.

18. "My agent loops infinitely."

- **Root cause:** A cyclic state in the orchestration graph (e.g. planner re-invokes itself, or a retry loop with no termination condition) that never reaches a terminal state.
- **Diagnosis:** Check the orchestrator's execution trace for a repeating state pattern — this is exactly the kind of failure an explicit state machine (LangGraph-style, or Step Functions on AWS) makes visible, versus a plain recursive chain where it's much harder to detect.
- **Production solution:** Every cyclic path in the orchestration graph needs an explicit termination condition — a maximum retry/iteration count, independent of whether the underlying logic "should" converge. This is standard practice for any state machine with cycles, not unique to LLM agents, but easy to omit when the cycle is meant to handle retries/re-planning.
- **Monitoring:** Alert on any single request's execution time or step count exceeding a sane ceiling — a request still running after 60 seconds when the p99 is 3 seconds is a strong infinite-loop signal.
- **Prevention:** Enforce a hard iteration cap at the orchestration framework level (not just application logic), so a bug in one node's exit condition can't produce an unbounded loop.
- **Best practice:** Any design that supports cycles for legitimate reasons (retries, re-planning) needs an explicit, framework-enforced ceiling — "the logic should terminate" is not a safety guarantee.

19. "My orchestration fails midway."

- **Root cause:** One step in the hybrid flow (e.g. the correlation step, or the synthesis call) throws an exception after earlier steps (SQL execution, retrieval) already succeeded.
- **Diagnosis:** Check which step failed and whether the earlier steps' results are still available — this is exactly why the hybrid orchestrator's steps are explicit and separable (Section 10) rather than one opaque black box; a failure at step 6 (correlate) doesn't mean steps 3-5 (SQL, retrieval, fusion) need to be redone.

- **Production solution:** Step Functions (on AWS) or a similarly durable state machine persists intermediate state, so a mid-flow failure can retry from the failed step rather than from the beginning — expensive completed work (SQL execution, retrieval) isn't wasted on a retry.
 - **Monitoring:** Per-step success/failure rates within the orchestration flow, not just an aggregate "hybrid query failure rate" — this pinpoints which step is actually unreliable.
 - **Prevention:** Idempotent step design (a retried step shouldn't have side effects that compound, e.g. writing to cache/audit twice) so retries are safe by default.
 - **Best practice:** Durable, resumable orchestration (not a single Python process holding all state in memory) is what turns "one step failed" into "retry that step" instead of "the whole request failed and the user gets nothing."
-

Part B — CDWIA-specific question bank

Beginner (15)

- 1. What does CDWIA stand for and what problem does it solve?** Cloud Data Warehouse Intelligence Assistant — it lets business users ask natural-language questions about cloud cost/usage data and get answers that combine structured SQL results with FinOps documentation, instead of requiring SQL knowledge or manually cross-referencing dashboards with best-practice docs.
- 2. What are the three query paths the system can take?** SQL (structured data questions), document/knowledge retrieval (policy/best-practice questions), and hybrid (questions needing both — e.g. "why did cost increase and what should we do about it").
- 3. Why doesn't every question go through an LLM to decide the query path?** Because most questions are structurally obvious and a lightweight deterministic classifier can route them in milliseconds without the latency and dollar cost of an LLM call; the LLM is reserved as a fallback for genuinely ambiguous cases.
- 4. What is the control plane, and what is the data plane?** The control plane makes decisions — authentication, authorization, classification, cost checks. The data plane executes — running SQL and retrieving documents. The control plane always sits before the data plane in the request path so denied/invalid requests are rejected before any expensive execution happens.
- 5. What does the AST SQL validator do?** Parses every LLM-generated SQL string into an abstract syntax tree and checks it against a five-step decision tree — parseable, SELECT-only, tables/columns on an allowlist, has a LIMIT (or gets one injected), and under a cost threshold (or gets queued async) — before it's allowed to execute.
- 6. Why use an AST parser instead of just checking for forbidden keywords in the SQL string?** Because keyword/regex filtering operates on surface text, which can be

manipulated with comments, string tricks, or unusual formatting — an AST reflects the query's actual parsed structure, which can't be disguised the same way.

7. What is hybrid retrieval? Running both BM25 keyword search and vector semantic search on the same query, then combining (fusing) the results — because BM25 catches exact terms (like account IDs) that vector search can miss, and vector search catches semantic intent that keyword matching alone misses.

8. What's the difference between authentication and authorization in this system? Authentication (handled by the gateway) answers "who is this" from a token. Authorization (handled by the control plane's policy engine) answers "can this identity do this specific thing" based on role, resource, and policy — deliberately kept separate so policy changes don't touch the login code path.

9. Why is there a cache, and what's special about how it's keyed? To avoid recomputing the full pipeline for repeated questions. It's keyed on the normalized query text *plus* the requester's data scope (tenant/business unit) — never on query text alone — so a cached answer can never leak across a scope boundary it shouldn't.

10. What is citation enforcement? A structural check that every factual claim in the synthesized answer is tagged with a citation to something actually retrieved or executed — any citation that doesn't resolve to real evidence causes the answer to be rejected, rather than trusting the model's prompt compliance alone.

11. Why does the SQL executor use a read-only database role even though the SQL was already validated? Defense in depth — the validator is application code and can have bugs; a read-only role enforced by the database itself means even a validator bug can't result in a write, because the database is structurally incapable of it regardless of what SQL reaches it.

12. What's the purpose of the audit log, and why does it need to be immutable? It records what was asked, what SQL ran, what was cited, and whether guardrails passed — needed to answer "what did the system know when it made this recommendation" for compliance and debugging. It's immutable (no update/delete permissions) so it can't be altered after the fact, which is what makes it trustworthy as a record.

13. What does the guardrail pass check that citation enforcement doesn't? Citation enforcement verifies a cited source *ID* is real. The guardrail's hallucination check goes further — it verifies the *claim itself* actually follows from that source's content, catching cases where a real citation is attached to an inaccurate paraphrase.

14. Why is ingestion a separate, asynchronous pipeline instead of happening inline with queries? Because ingestion (processing new documents/data) and live query serving have completely different latency requirements — ingestion can tolerate delay, live queries can't. Coupling them would let a burst of new documents slow down user-facing queries for no good reason.

15. What happens if the SQL path fails validation during a hybrid query? The hybrid orchestrator doesn't fail the whole request — it proceeds using the document retrieval

results alone, with a warning attached, so the user still gets a partial, honest answer rather than nothing.

Intermediate (20)

1. Walk through what happens end-to-end for the query "Our storage cost increased by 45%, how can we optimize it?" The gateway authenticates the request; the deterministic classifier scores it ambiguous (mixed cost + optimization signals) and falls back to the LLM planner, which confirms "hybrid"; the orchestrator fans out SQL generation (validated, executed read-only) and document retrieval (BM25+vector, fused, reranked) in parallel; results are merged and correlated by a recommendation step; the synthesizer produces a citation-enforced answer; guardrails re-check for hallucination and PII; the result is cached (scoped) and audit-logged before returning to the user.

2. Why is the classifier's confidence threshold tunable rather than fixed? Because the right threshold depends on the tradeoff between false-confident misroutes (bad if threshold is too low) and unnecessary LLM fallback cost (bad if threshold is too high) — this is tuned empirically against a labeled dataset of real queries, and that tradeoff point shifts as the classifier itself improves or as traffic patterns change.

3. What's the risk of caching hybrid query answers, and how is it mitigated? The risk is serving stale data after underlying billing data or documents change. It's mitigated with a TTL (short enough that staleness is acceptable) and, in a more precise design, tagging cache entries with the data version they were computed against so they can be invalidated when that version is superseded.

4. Why does reciprocal rank fusion make sense for combining BM25 and vector search results, when a weighted average of scores wouldn't? BM25 and cosine-similarity scores are on incomparable scales — there's no principled way to weight-average an 8.2 against a 0.82. RRF sidesteps this entirely by fusing based on each result's *rank position* within its own list, which is scale-independent.

5. What's the failure mode if the OPA/policy engine becomes unreachable, and why was that choice made deliberately? The system fails closed — every request is denied, not allowed. This is deliberate because the alternative (failing open) means a policy engine outage becomes a silent total access-control bypass, which is a far worse failure mode than temporarily blocking legitimate users.

6. Why is the SQL cost estimator based on EXPLAIN output rather than a fixed row-count limit? A row-count limit doesn't capture query cost accurately — a query with a small result set can still be catastrophically expensive if it involves a large scan or an unindexed join. An EXPLAIN-based estimate reflects the actual planned execution cost, which is what threatens the warehouse.

7. How does the model router's fallback chain change the blast radius of a provider outage? Instead of the whole system failing when the preferred model is unavailable, the router walks down an ordered chain to the next available model — an outage degrades answer quality one tier, rather than taking availability down entirely.

8. Why does the design separate "merge" and "synthesize" and "guardrail-check" into distinct steps instead of one "LLM synthesizes" step? Each step can fail independently and needs distinct error handling — a merge/correlation bug is a different problem than a synthesis hallucination, which is different again from a guardrail catching a PII leak. Collapsing them into one opaque step would mean one failure mode masks the others and makes debugging much harder.

9. What's the tradeoff of injecting a default LIMIT versus rejecting queries that lack one? Rejecting harms the user experience for what's usually an innocent oversight in generation, not a security problem. Injecting a sane default preserves usefulness while still protecting against unbounded result sets — the design treats "missing LIMIT" as a correctable mistake, not a violation.

10. Why does contextual compression matter for both cost and hallucination risk? Fewer, more relevant tokens in context directly reduces per-call token cost, and also reduces the amount of irrelevant text the model could misuse to construct a plausible-but-wrong claim — both the economic and the quality argument point the same direction.

11. How would a schema change (a column renamed or removed) affect this system, and what would break first? The SQL allowlist config would reference a column that no longer exists, causing the AST validator to reject any query referencing it once regenerated SQL tries to use the new name (which isn't in the allowlist yet) — this would surface as a spike in `REJECT_OUT_OF_SCOPE` rejections, which is the first signal something changed, well before it becomes a silent wrong-answer problem.

12. Why is the classifier's low-confidence default set to "hybrid" rather than, say, "SQL"? Hybrid tends to give supplementary coverage — worst case, it runs an extra retrieval or SQL path that returns nothing useful, which is a safer failure mode than confidently routing an ambiguous question down a single path and missing half the answer.

13. What's the actual authorization boundary for SQL query scope, and why isn't RLS alone sufficient? The AST validator's allowlist is the primary authorization boundary — it decides what's allowed to even be attempted, described as something to review "like an IAM policy." RLS is a critical backstop, but relying on it alone means a query that's technically read-only and in-scope by the DB role's permissions could still be catastrophically expensive or violate an intent-level scoping rule the DB role doesn't encode.

14. Why does the design use dependency-injected interfaces (e.g. Python `Protocol` classes) throughout, rather than hardcoding specific backends? It makes swapping or extending backends (a new vector store, a new warehouse) additive rather than invasive — you implement the interface for the new backend without touching the core decision logic (the validator, the orchestrator, the classifier) that depends only on the contract, not the implementation.

15. Why does the design explicitly call out "no string/regex-only filtering" for both SQL safety and prompt safety? Because both are the same underlying failure pattern — surface-text filtering can be bypassed by anything that changes the text without changing

the underlying meaning (comments in SQL, clever phrasing in a prompt injection attempt). Structural checks (AST parsing, or more principled input/output scanning) don't have this blind spot.

16. What would you monitor to catch a slow degradation in the classifier's accuracy over time? A rising LLM-fallback invocation rate (more questions falling below the confidence threshold than historically), combined with periodic manual review of a sample of classifier decisions against ground truth — since classifier "accuracy" isn't directly observable in production without labeled data, fallback rate and sampled review are the practical proxies.

17. Why is "queue as async job" a valid outcome for the SQL validator, rather than the query simply failing? Because a legitimate, correctly-scoped query can still be too expensive to run inline — rejecting it outright means a valid question goes unanswered, while queuing it preserves usefulness (the user gets a real answer, just not instantly) without letting the query threaten the warehouse.

18. How does the system prevent a cross-tenant data leak through the result cache? By including the requester's tenant/business-unit scope as part of the cache key hash, not as a filter applied after a cache read — this makes cross-scope leakage structurally impossible rather than dependent on remembering to check scope on every cache access.

19. What's the relationship between the semantic layer and the SQL allowlist? The semantic layer governs what a metric like "cost" *means* (which tables/columns it maps to); the allowlist governs what the SQL agent is *authorized* to touch. They're complementary — the semantic layer solves correctness/consistency of meaning, the allowlist solves security/scope.

20. Why does the design prefer redacting detected PII in an outbound response over rejecting the whole answer? Rejecting the whole answer discards otherwise-useful information over one sensitive field; redaction protects the sensitive data while preserving the value of the rest of the answer — a better tradeoff for an assistant than a strict fail-closed compliance gate that fails on any imperfection.

Advanced (20)

1. The classifier and the LLM planner can disagree. Design a mechanism to detect when the classifier is systematically wrong for a specific query pattern, without waiting for manual review. Log every case where the LLM planner overrides the classifier (`used_llm_fallback=True`), and periodically cluster those queries (e.g. by embedding similarity or keyword pattern) — a cluster of similar queries repeatedly triggering fallback and getting the same LLM verdict is a strong, automatable signal that the classifier should be retrained or its rules extended to cover that pattern deterministically, closing the gap without needing every case manually reviewed.

2. If you had to support real-time (sub-second) cost updates instead of the current async ingestion model, what would break, and how would you redesign around it? The core assumption that breaks is "ingestion can tolerate delay" — real-time updates mean the semantic layer's cached/materialized cost aggregates go stale immediately. You'd likely

need to separate "hot" recent data (queried directly against a streaming-updated table or a fast OLTP store) from "warm/cold" historical data (still served from the OLAP warehouse), with the SQL agent's semantic layer routing queries by time range — recent-window queries hit the fast path, historical aggregation queries hit the warehouse. This adds real complexity (two storage systems to keep consistent) that the current design deliberately avoids by accepting near-real-time freshness instead.

3. The design uses a circuit breaker around SQL execution. Design the state transition logic in detail, including what "half-open" should actually test. Closed state: requests pass through normally, consecutive failures tracked. On reaching a failure threshold, transition to open: fail fast for a cooldown window, no requests reach the warehouse. After cooldown, transition to half-open: allow exactly one (or a small bounded number of) trial request(s) through — critically, this trial should be a cheap, known-safe query (e.g. `SELECT 1`), not an arbitrary real user query, so a slow-but-not-dead warehouse doesn't get re-opened prematurely by one lucky fast query, and a genuinely-recovered warehouse isn't kept closed by one unlucky slow one. If the trial succeeds, close the circuit; if it fails, return to open and extend the cooldown (exponential backoff on the cooldown window itself, not just the retry).

4. Citation enforcement raises an error on an unsupported claim. In a production system, what should actually happen next, end-to-end, and what are the tradeoffs of each option? Three options: (a) retry synthesis with a stricter prompt — costs latency and another model call, no guarantee of success; (b) strip the unsupported claim and re-verify the remainder — preserves partial value but requires the synthesizer to support incremental re-verification, more implementation complexity; (c) fail the request with a "couldn't produce a fully-cited answer" response — simplest, safest, worst user experience. Production systems typically implement (a) with a retry cap, falling back to (c) if retries are exhausted — this bounds latency cost while still trying to recover before giving up entirely.

5. The vector store is namespaced by tenant for multi-tenancy. What happens to retrieval quality for a tenant with very little indexed content, and how would you address it without breaking isolation? A tenant with a small corpus gets a small candidate pool for BM25/vector fusion — reranking has less to work with, and precision suffers not because the pipeline is broken but because there's genuinely less relevant content to find. You cannot pull in another tenant's namespace without breaking isolation, so the fix has to be either (a) a shared, tenant-agnostic corpus of genuinely universal content (e.g. AWS Well-Architected Framework, FinOps Foundation guidance) indexed separately and merged into every tenant's retrieval results at query time, distinct from tenant-private content — giving small tenants a useful baseline without leaking any tenant-specific data across the boundary, or (b) accepting the lower-relevance result and letting the confidence score reflect it honestly.

6. Design the golden dataset evaluation pipeline referenced in the roadmap. What should it actually test, and how do you prevent it from becoming stale? It should test at minimum: classifier routing accuracy (question → expected path), SQL generation correctness (question → expected SQL or expected result set, not just "did it execute"),

retrieval precision@k on a labeled question→relevant-chunk set, and end-to-end citation validity on hybrid queries. Run it in CI on every prompt, model, or schema change — not just on a schedule — so regressions are caught before deployment, not after. Staleness is the real long-term risk: the golden dataset needs its own ownership and update cadence, ideally seeded partly from real production queries that were manually verified correct, so it evolves with actual usage patterns rather than staying frozen at whatever the team thought to test at launch.

7. Compare the security posture of "AST validation + allowlist + read-only role + RLS" against a design that instead uses a fully sandboxed, ephemeral read-replica per query. What does each approach actually protect against that the other doesn't? The layered design (AST + allowlist + role + RLS) protects against *malicious or buggy SQL reaching production data it shouldn't*, at low marginal cost per query (no per-query infrastructure). An ephemeral-replica-per-query design additionally protects against *resource contention* — a runaway query can't affect any other tenant's queries because it's isolated to its own replica — but at significant cost and latency overhead (spinning up a replica per query is far from free), and it still needs the same authorization/scope checks, since an ephemeral replica with full data access and no RLS is just as leaky as a shared one. In practice, the layered approach is right for most workloads; ephemeral isolation is worth its cost only for genuinely adversarial or extremely noisy-neighbor-prone multi-tenant environments.

8. The hybrid orchestrator runs SQL generation and retrieval concurrently. Under what circumstance would this concurrency actually hurt rather than help, and how would you detect it? If the underlying compute (Lambda concurrency limits, warehouse connection pool, embedding API rate limits) is shared and constrained, running both paths concurrently for every hybrid request doubles simultaneous resource pressure compared to sequential execution — under high load, this could cause more queuing/throttling than the latency it saves. You'd detect this by comparing p99 latency for hybrid queries under load against p99 for sequential execution in a load test, and by watching for correlated throttling/error spikes specifically during hybrid-heavy traffic bursts, not just overall traffic bursts.

9. If you needed to support an SLA of "answers must never be more than 5 minutes stale relative to the source data," how would you change the caching and ingestion design? Cache TTL would need to be bounded well under 5 minutes, or better, tied to actual data freshness rather than a flat TTL — tag cache entries with the data version/timestamp they were computed against, and invalidate proactively when ingestion completes a new version rather than waiting for TTL expiry. Ingestion pipeline lag itself (time from source change to index/warehouse availability) becomes a hard SLA component that needs its own monitoring and alerting, since a 5-minute end-to-end guarantee is only as good as the slowest link — if ingestion itself can lag 4 minutes, there's only 1 minute of budget left for caching and query latency combined.

10. The design doc recommends cost-per-query broken down by SQL/RAG/hybrid path as a first-class metric. Design the actual instrumentation — what has to be tagged, where, and what's hard about getting this right? Every model call (classifier fallback, SQL

generation, synthesis, reranking) needs to emit its token count and model tier, tagged with the originating request ID and which path (SQL/RAG/hybrid) it belongs to; every SQL execution needs its EXPLAIN-estimated cost tagged the same way. These all need to roll up to a single per-request total, aggregated at the audit-log or a dedicated cost-tracking layer. The hard part: hybrid queries genuinely span both SQL and RAG cost — attributing cost cleanly to "the hybrid path" as a whole (rather than trying to split it artificially between SQL-cost and RAG-cost components) is usually more useful than forcing an artificial split, but that requires the instrumentation to model "hybrid" as its own first-class cost category from the start rather than bolting it on as a sum of the other two after the fact.

11. Why might a naive implementation of reciprocal rank fusion still produce poor results even though the math is scale-independent, and what would you check first?

RRF is scale-independent across the two lists it fuses, but it's still sensitive to each individual list's quality — if BM25 is returning near-random results for a given query (e.g. because the query has no exact-match terms at all), RRF will still weight BM25's "top" (effectively noise) results at their rank position, diluting the vector search's genuinely relevant results. Check first whether the failing queries have exact-match signal at all — if not, the fusion weighting (or the $\frac{1}{k}$ constant in RRF) may need to be more dynamic, favoring vector search more heavily when BM25's own result quality is low, rather than always trusting both lists equally.

12. Design a mechanism to detect embedding drift automatically, without waiting for a human to notice degraded retrieval quality. Maintain a small, stable set of canary query→expected-chunk pairs, re-run them on a schedule against the live index, and track precision@k over time — a sudden drop coincides with either an embedding model change that wasn't accompanied by full reindexing, or index corruption. This is cheaper and faster than waiting for aggregate user-facing relevance metrics to visibly degrade, since canary queries are deliberately chosen to be stable and unambiguous.

13. The design fails closed on policy engine unavailability. Should the SQL cost estimator also fail closed (queue everything) or fail open (execute everything) if the estimator itself becomes unavailable? Justify your answer. Fail closed (queue, don't execute) — the cost estimator's entire purpose is preventing warehouse-locking queries from running unchecked; if it's unavailable, you have no signal on which queries are safe, and "assume everything is safe" is exactly the scenario the estimator exists to prevent. The cost of failing closed here (some legitimate queries get delayed to async processing during the estimator's outage) is far smaller than the cost of failing open (a single catastrophic query executing inline during exactly the window when nothing is checking for it).

14. How would you extend the AST validator to detect and reject queries with unbounded cartesian products from joins missing an ON clause, and what's the risk of getting this rule wrong? Walk the parsed AST's `Join` nodes and check for a missing or trivially-true `on` condition (e.g. `ON 1=1`), rejecting or flagging those. The risk of getting it wrong: false positives on legitimate cross joins used deliberately (rare but real, e.g. generating a cartesian product of a small dimension table intentionally) — an overly strict rule blocks a valid use case. Mitigation: make this a cost-estimator concern as much as a

validator concern — a deliberate small cross join is cheap and the EXPLAIN-based cost check would pass it fine, while an accidental one on large tables is caught by cost regardless of whether the join-condition rule flags it, giving you two independent nets rather than relying on a brittle single rule.

15. If the reranker (cross-encoder) itself becomes a latency bottleneck under load, what are your options, ranked by implementation cost? Cheapest: reduce the candidate set size going into reranking (tighten fusion's `top_k` before reranking, so the expensive step processes fewer pairs). Next: cache rerank scores for repeated (query, chunk) pairs, since popular queries against a relatively stable corpus will re-rerank the same pairs often. More involved: use a smaller/faster reranker model for latency-sensitive traffic and reserve the highest-quality reranker for a background "refine" pass whose results get cached for next time (serve slightly-lower-quality-but-fast now, upgrade quality for future identical queries later). Most involved: batch rerank requests across concurrent queries to amortize the cross-encoder's fixed overhead, if the reranking service supports batched inference.

16. Explain how you'd design a canary/shadow deployment for a new model version in the router's fallback chain, without risking production answer quality. Add the new model as a shadow call alongside (not replacing) the current production model — every request triggers both, but only the production model's output is returned to the user; the shadow model's output is logged and scored (against citation validity, guardrail pass rate, and ideally a sampled human/automated quality comparison) without affecting anyone's actual experience. Once the shadow model's metrics meet or beat the current model's over a sufficient sample size and time window, promote it — first as a low-percentage live canary (real traffic, real answers, small blast radius), then to full production once the canary window confirms no regression.

17. The design doc's cost levers include "context compression reducing token spend." What's the risk of over-compressing, and how would you detect you've gone too far? Over-compression risks removing context the synthesizer actually needed to answer accurately or to construct a valid citation — you'd see this as a rise in citation-enforcement failures or hallucination-guardrail failures correlating with a compression-aggressiveness change, even though token cost per call dropped. The detection mechanism is the same golden-dataset eval pipeline used elsewhere: track answer quality/citation-validity metrics alongside token cost, and treat any compression tuning as a two-metric optimization (cost down, quality flat-or-up), not a single-metric one.

18. Design a strategy for safely rolling out a change to the SQL allowlist (adding a new table) without risking either over-exposure or breaking existing query generation. Stage the allowlist change behind the same kind of canary approach as a model rollout — add the new table/columns to a non-production allowlist config first, run the golden dataset (extended with new test cases exercising the new table) against it, and review the diff of what queries the SQL generator now produces with the new table available (since a new table appearing in scope can change how the LLM resolves ambiguous questions, not just add new capability). Only promote to the production allowlist after this review, and treat

the allowlist file the same way the design doc already recommends — reviewed like an IAM policy change, with a clear diff and sign-off, not a silent config push.

19. If two different business units need genuinely different SQL safety policies (one stricter than the other), how would you extend the current single-allowlist design to support that without duplicating the validator logic? Make the allowlist scoped per business unit/tenant rather than global — the `SchemaAllowlist` the validator consumes becomes a function of the requester's `Principal` (already carrying `business_unit`), loaded per-request or cached per-scope, rather than one global config. The validator's decision-tree logic itself doesn't change at all — only which allowlist instance it's checking against — which is exactly the value of having kept the validator's structural logic separate from the allowlist's data in the first place.

20. You're asked to justify, to a security review board, why "read-only role + RLS" is sufficient without also requiring per-query manual approval for anything touching cost/billing data. Make that case, and note where you'd concede a human-approval step is warranted. The case: automated layered defense (AST validation, allowlist, read-only role, RLS, cost estimation, audit logging) provides consistent, testable, always-on protection that a human reviewer — inherently inconsistent, slow, and a bottleneck at scale — cannot match for the high-volume, low-individual-risk case of read-only analytical queries. Where you'd concede human approval is warranted: any SQL path that could plausibly touch write operations (none should exist here, by design), any query the cost estimator flags as exceeding even the async-queue threshold by a wide margin (potential signal of a probing/abuse pattern worth a human look), and any change to the allowlist or RLS policy itself (a configuration change, not a query, and configuration changes to the security boundary are exactly where human review adds real value versus per-query approval, which doesn't scale and doesn't actually catch what automated layers already catch better).

Part C — Cross-examination drills

This is the "keep drilling" format from your brief. Practice extending these yourself on every other decision in the design doc (LangGraph, Redis, OPA, sqlglot, Kafka, etc.) using the same pattern: candidate answer → increasingly specific "why not X" → failure-mode questions → operational questions.

Drill 1: "I used a hybrid vector store (OpenSearch Serverless / Pinecone) for retrieval."

Why not FAISS? FAISS is a library, not a managed service — no built-in persistence, replication, or multi-tenant namespacing. You'd be building the operational layer (backup, HA, scaling) yourself on top of it, which is exactly the burden a managed vector store removes.

Why not pgvector, since you're already running Postgres/a relational warehouse?

pgvector is a reasonable choice at smaller scale and keeps everything in one database, which simplifies operations. It was not chosen here because at the scale this system targets (potentially hundreds of millions of embeddings), a purpose-built vector store's ANN

indexing and horizontal scaling outperform pgvector's approach, and coupling vector search load onto the same database serving SQL query traffic risks resource contention between two very different workload types.

Why not Milvus or Weaviate (self-hosted alternatives)? Both are capable, but self-hosting either means owning cluster operations (scaling, backup, HA) — the same operational-burden argument as FAISS, just at a more capable tier. A managed option was preferred specifically to avoid that ongoing operational cost, given the team's priority was shipping the RAG pipeline, not operating a vector database cluster.

What if the vector store fails? Covered in Scenario 1 — the circuit breaker isolates the failure, and hybrid queries degrade to SQL-only rather than failing entirely.

What if latency increases? The knowledge agent's retrieval call should have its own timeout, separate from the overall request timeout, so a slow vector store degrades gracefully (partial results, or a fallback to BM25-only) rather than blocking the whole hybrid flow.

How do you migrate vectors to a different provider if needed? Since chunk text and metadata are the source of truth (stored separately, per the ingestion pipeline's "object storage first" design), migration means re-embedding from that source of truth into the new store — you're never dependent on being able to export vectors directly from the old provider, which is a real risk with some managed offerings.

How do you back up vectors? Given the above, backup is really about backing up the source text/metadata (standard object storage backup practices) plus periodic embedding-model-versioned snapshots if the provider doesn't offer native backup — the vectors themselves are reconstructable from source, which is a stronger position than needing point-in-time vector-store backups.

How do you monitor vector quality over time? Precision@k / recall@k against a labeled canary query set, run on a schedule (as in the embedding-drift scenario) — this catches both index corruption and slow relevance degradation from corpus drift.

How do you reduce retrieval latency if it becomes a bottleneck? Reduce top-k before fusion/reranking, cache frequent query embeddings, consider a faster/approximate ANN configuration if the store supports tuning the recall/latency tradeoff, and check whether reranking (not the vector search itself) is actually the bottleneck — the earlier reranker-latency drill answer applies here too.

Drill 2: "I used a deterministic classifier before the LLM planner for query routing."

Why not just always use the LLM — it's more flexible? Flexibility isn't free — every request would pay the latency and token cost of an LLM call just to route, even for the majority of questions that are structurally unambiguous. The classifier captures nearly all of that value at a fraction of the cost.

Why not a larger, more accurate classifier model instead of regex/lightweight heuristics? The reference implementation uses regex as a stand-in; production should use a small

trained model behind the same interface — the point isn't "regex forever," it's "keep this step cheap and fast," which a small trained classifier still satisfies while regex alone eventually would not, once query patterns get more varied than simple keyword rules can capture.

What if the classifier is confidently wrong? Downstream agents are the safety net — a wrongly-routed SQL query still goes through the AST validator; a wrongly-routed document query into the hybrid path just runs a (harmlessly empty-ish) SQL path alongside retrieval. The system degrades the answer rather than executing something unsafe.

How do you retrain/update the classifier without downtime? Since it's behind a stable interface (`ClassificationResult` with path + confidence), a new model version can be validated offline against the golden dataset, then deployed as a drop-in replacement — no changes needed anywhere else in the control plane that consumes its output.

How do you know the confidence threshold is still well-calibrated over time? Track LLM-fallback rate as a leading indicator — a rising rate over time, without a corresponding rise in genuinely ambiguous query volume, suggests the classifier's confidence calibration has drifted and needs re-tuning against fresh labeled data.

What if traffic patterns shift and introduce a whole new category of question the classifier has never seen? Novel patterns naturally produce low classifier confidence (nothing in its rules/training matches), which correctly triggers LLM fallback — this is the designed behavior for exactly this case, not a failure. The classifier only needs retraining once that new pattern becomes common enough that paying the LLM-fallback cost for it repeatedly is worth optimizing away.

How would you A/B test a new classifier version safely? Shadow it first (same pattern as the model canary drill) — run the new classifier alongside the production one on live traffic, log both decisions, compare against ground truth on a sampled/labeled subset, and only cut over once its accuracy and calibration are verified to meet or beat the current version.

Why is confidence scoring important here specifically, versus just returning a hard classification? Confidence is what makes the fallback mechanism possible at all — without it, there's no principled way to decide which requests deserve the more expensive LLM's judgment versus which the classifier's direct answer can be trusted for.

Reply with which installment to build next: the technical-terms glossary, expanded question bank (to reach the full 30/50/70), coding questions, system design questions, or the 100-question mock interview.